



Universidad
Rafael Landívar
Tradicón Jesuita en Guatemala

Universidad Rafael Landívar

Campus Quetzaltenango

Licenciatura en Ingeniería en Informática y Sistemas

Estructuras de Datos I 2026

Catedrático: Ing. Juan Pablo Valiente

Manual Técnico

Proyecto final: Generador de imágenes por capas

Nombre: Pablo Ernesto Castro Fuentes

Carné: 1597825

Fecha: 29/05/2026

1. Introducción

El presente manual técnico describe la estructura interna del sistema Generador de imágenes por capas, desarrollado en lenguaje C++. El programa utiliza estructuras dinámicas para almacenar capas, imágenes y usuarios, así como para generar reportes visuales del estado de memoria mediante Graphviz.

2. Lenguaje y herramientas utilizadas

- Lenguaje de programación: C++.
- Compilador: g++ mediante MinGW.
- Entorno de desarrollo: Visual Studio Code.
- Sistema operativo: Windows.
- Herramienta de visualización: Graphviz.
- Formato de salida de imágenes: HTML.

3. Archivos principales del proyecto

```
main.cpp  
capas.cap  
imagenes.im  
usuarios.usr  
proyecto.exe
```

El programa también genera archivos HTML, archivos DOT y archivos PNG como evidencia de la generación de imágenes y reportes.

4. Descripción general del sistema

El sistema carga información desde archivos externos y la almacena en estructuras dinámicas. Las capas se almacenan en un árbol binario de búsqueda, las imágenes en una lista circular doble y los usuarios en otro árbol binario de búsqueda. Cada imagen contiene una lista de capas y cada usuario contiene una lista de imágenes.

5. Estructuras utilizadas

5.1 NodoPixel

Representa cada píxel de una capa. Almacena fila, columna, color y un puntero al siguiente píxel.

```
struct NodoPixel {
    int fila;
    int columna;
    string color;
    NodoPixel* siguiente;
};
```

5.2 NodoCapa

Representa una capa dentro del árbol binario de búsqueda. Contiene su ID, una lista de píxeles y punteros a los hijos izquierdo y derecho.

```
struct NodoCapa {
    int idCapa;
    NodoPixel* pixeles;
    NodoCapa* izquierda;
    NodoCapa* derecha;
};
```

5.3 NodoCapaImagen

Representa un nodo dentro de la lista de capas de una imagen. Cada nodo apunta a una capa existente en el árbol de capas.

```
struct NodoCapaImagen {
    NodoCapa* capa;
    NodoCapaImagen* siguiente;
};
```

5.4 NodoImagen

Representa una imagen dentro de la lista circular doble. Contiene el ID de la imagen, su lista de capas y enlaces al nodo siguiente y anterior.

```
struct NodoImagen {
    int idImagen;
    NodoCapaImagen* listaCapas;
    NodoImagen* siguiente;
    NodoImagen* anterior;
};
```

5.5 NodoImagenUsuario

Representa una imagen asignada a un usuario. Cada nodo apunta a una imagen existente en la lista circular de imágenes.

```
struct NodoImagenUsuario {
    NodoImagen* imagen;
    NodoImagenUsuario* siguiente;
};
```

5.6 NodoUsuario

Representa un usuario dentro del árbol binario de búsqueda de usuarios. Contiene el nombre, una lista de imágenes y punteros a sus hijos.

```
struct NodoUsuario {
    string nombre;
    NodoImagenUsuario* listaImagenes;
    NodoUsuario* izquierda;
    NodoUsuario* derecha;
};
```

6. Carga masiva de datos

6.1 Carga masiva de capas

La función `cargaMasivaCapas()` lee el archivo `capas.cap`. Por cada capa encontrada, inserta un nodo en el árbol binario de búsqueda y agrega sus píxeles en una lista enlazada simple.

6.2 Carga masiva de imágenes

La función `cargaMasivaImagenes()` lee el archivo `imagenes.im`. Por cada imagen, crea un nodo en la lista circular doble y enlaza las capas correspondientes mediante punteros hacia el árbol de capas.

6.3 Carga masiva de usuarios

La función `cargaMasivaUsuarios()` lee el archivo `usuarios.usr`. Por cada usuario, inserta un nodo en el árbol de usuarios y asigna las imágenes indicadas en el archivo.

7. Árbol binario de búsqueda de capas

Las capas se almacenan en un árbol binario de búsqueda ordenado por el ID de la capa. Esta estructura permite insertar, buscar y recorrer capas en preorden, inorden y postorden.

```
insertarCapa()  
buscarCapa()  
mostrarCapasInorden()  
mostrarCapasPreorden()  
mostrarCapasPostorden()
```

8. Lista circular doble de imágenes

Las imágenes se almacenan en una lista circular doble. Cada imagen posee punteros al nodo siguiente y anterior. La última imagen apunta a la primera y la primera apunta a la última.

```
crearNodoImagen()  
insertarImagenCircular()  
buscarImagen()  
eliminarImagenCircular()  
mostrarListaImagenes()
```

9. Árbol binario de búsqueda de usuarios

Los usuarios se almacenan en un árbol binario de búsqueda ordenado por el nombre del usuario. Esta estructura permite insertar, buscar, modificar y eliminar usuarios.

```
insertarUsuario()  
buscarUsuario()  
eliminarUsuarioABB()  
mostrarUsuariosInorden()
```

10. Generación de imágenes

El sistema genera imágenes mediante archivos HTML. La versión final utiliza matrices dinámicas cuyo tamaño se calcula automáticamente a partir de los píxeles almacenados en las capas. Esto permite trabajar con archivos grandes sin limitarse a una matriz fija de 10 x 10.

```
calcularTamanoImagen()  
calcularTamanoCapa()  
calcularTamanoArregloCapas()  
generarImagenHTML()  
generarCapaHTML()  
generarImagenPorArregloCapas()
```

10.1 Generación por lista de imágenes

La función `generarImagenPorLista()` busca una imagen por ID y llama a `generarImagenHTML()` para producir el archivo `imagen_generada.html`.

10.2 Generación por capa

La función `generarImagenPorCapa()` busca una capa por ID y llama a `generarCapaHTML()` para generar el archivo `capa_generada.html`.

10.3 Generación por usuario

La función `generarImagenPorUsuario()` valida que el usuario exista, que tenga imágenes asignadas y que la imagen solicitada pertenezca a dicho usuario.

10.4 Generación por recorrido limitado

La generación por recorrido limitado utiliza el árbol de capas y permite seleccionar preorden, inorden o postorden. Luego genera la imagen utilizando las capas recolectadas en ese orden.

```
recolectarPreorden()  
recolectarInorden()  
recolectarPostorden()  
generarImagenPorArregloCapas()
```

11. CRUD de usuarios

El sistema permite administrar usuarios mediante un menú específico. Las operaciones incluyen agregar, buscar, mostrar, modificar y eliminar usuarios. También se puede agregar o eliminar una imagen asignada a un usuario.

```
menuCRUDUsuarios()  
modificarNombreUsuario()  
eliminarUsuarioDesdeCRUD()  
eliminarImagenDeUsuarioDesdeCRUD()
```

12. CRUD de imágenes

El sistema permite administrar imágenes mediante un menú específico. Se pueden mostrar imágenes cargadas, buscar imágenes, mostrar sus capas, agregar imágenes nuevas y eliminar imágenes.

```
menuCRUDImagenes()  
eliminarImagenDesdeCRUD()
```

13. Reportes con Graphviz

El sistema utiliza Graphviz para representar visualmente el estado de las estructuras de memoria. Se generan archivos .dot y .png.

```
generarGraphvizArbolCapas ()  
generarGraphvizArbolUsuarios ()  
generarGraphvizListaImagenes ()  
generarGraphvizImagenCapas ()
```

14. Manejo de archivos grandes

La versión final fue adaptada para archivos de capas grandes mediante matrices dinámicas, cálculo automático de filas y columnas, celdas pequeñas en HTML y contenedores desplazables. Además, los píxeles se insertan al inicio de la lista para mejorar el rendimiento durante cargas masivas extensas.

15. Flujo general del programa

```
Inicio del programa  
Mostrar menú principal  
Cargar capas  
Cargar imágenes  
Cargar usuarios  
Seleccionar operación  
Generar imagen, administrar datos o generar reporte  
Salir del programa
```

16. Conclusión técnica

El sistema cumple con los requerimientos funcionales del proyecto mediante el uso de árboles binarios de búsqueda, listas enlazadas, listas circulares dobles y matrices dinámicas. Además, integra Graphviz para representar estructuras de memoria y HTML para visualizar las imágenes generadas.